

Agile Principles

Lecture 03

1

12 Principles of Agile Software

- In addition to agile values, there are 12 principles that every agile practitioner should use when working on a software project team.
- When the Agile Manifesto was first formulated,
 - 4 values was finalized quickly.
 - 12 principles took its time.

<http://www.agilemanifesto.org/principles.html>

Agile Software Development

2

2

12 Principles of Agile Software

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Agile Software Development

3

3

12 Principles of Agile Software

4. The most efficient and effective method of conveying information to and within development team is face-to-face conversation.
5. Business people and developers must work together daily throughout the project.
6. Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.

Agile Software Development

4

4

12 Principles of Agile Software

- 7. Working software is the primary measure of progress.
- 8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- 9. Continuous attention to technical excellence and good design enhances agility.

Agile Software Development

5

5

12 Principles of Agile Software

- 10. Simplicity—the art of maximizing the amount of work not done—is essential.
- 11. The best architectures, requirements, and designs emerge from self-organizing teams.
- 12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile Software Development

6

6



**“If I had asked
people what
they wanted,
they would
have said
faster horses.”**
- Henry Ford

Agile Software Development

7

7

Customer Is Always Right...Right?

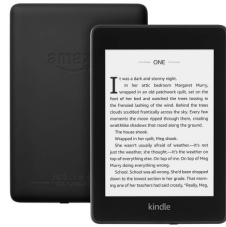
- Motivation behind the 12 principles is getting the team to build software that the user actually needs.
- The principles depend on the idea that we build projects to deliver **value**.
- Everyone sees different value in the software: different people need different things from it.

Agile Software Development

8

8

A handheld e-book reader



Agile Software Development

9

9

12 Principles of Agile Software

- To make it easier to learn, we've gathered these principles into four sections.
 - Delivery
 - Communication
 - Execution
 - Improvement
- Even though that's an effective way to learn about them, each principle stands on its own.

Agile Software Development

10

10

Delivering the Project

- Agile teams recognize that their most important job is to deliver working software to their customers.
- They do this by working together as a team, collaborating with their customers, and responding to change.
- But how does that translate to the day-to-day work of a team?



Agile Software Development

11

11

Delivering the Project

- Agile team does this by
 - Putting frequent delivery of value first.
 - Looking at each change as a good thing for the project.
 - Delivering software frequently.
- Team and the customers can work together to make adjustments along the way.

Agile Software Development

12

12

Agile Principle 1:

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.



Agile Software Development

13

13

Agile Principle 1

- Three distinct and important ideas:
 - Releasing software early
 - Delivering value continuously
 - Satisfying the customer
- It's impossible to completely and perfectly capture the requirements for any system.
- Until customers get their hands on working software, they can't imagine exactly how the software will work.

Agile Software Development

14

14

Agile Principle 1

- If the customer can only give real, informed feedback after seeing working software.
- Best way to get that feedback is through **early delivery**.
- Even if it is only a single working feature.
- Customer can now give informed feedback to help them move the project in the right direction.

Agile Software Development

15

15

Agile Principle 1

- Downside to delivering early is that the software that's first delivered to the customers is far from complete.
- When there's not a great partnership between the team and the clients, the users and stakeholders may judge it very harshly and even panic if a feature they expect to see is missing.

Agile Software Development

16

16

Agile Principle 1

- Core agile values have an answer to this issue.
 - Customer collaboration over Contract negotiation
- A team that truly collaborates with the customers has the option of making all the necessary changes along the way.
- That's what **continuous delivery** means.

Agile Software Development

17

17

Agile Principle 1

- This is why agile methodologies are typically iterative.
- Agile teams plan the project's iterations by choosing the features and requirements that will deliver the most value.
- Team satisfy the customer in the short run by demonstrating value early and in the long run by delivering the finished product that delivers as much value as possible.

Agile Software Development

18

18

Agile Principle 2:

Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.



Agile Software Development

19

19

Agile Principle 2

- Welcoming change is very difficult in real life project.
- Changes in project are emotionally charged.
- It's very difficult to accept someone telling you that you've been wrong.
- Especially if you've been doing work for that person.

Agile Software Development

20

20

Agile Principle 2

- Most software engineers are driven by pride of workmanship.
“We want to deliver products that we can stand behind, and that satisfy our users’ needs”
- A change in a project threatens this.
- Because it’s questioning the path that you’ve taken and the assumptions that you’ve made.

Agile Software Development

21

21

Agile Principle 2

- First step toward welcoming changing requirements is to try to see things from the customer’s perspective.
- Do you think that the customer who originally sent you down the wrong path meant to?



Agile Software Development

22

22

Agile Principle 2

- Both customer and software team is being asked to do the impossible.
- Team being asked to read the customer’s mind.
- Customer being asked to predict the future.
- When you look at it like that, it starts to become a lot easier to welcome change.

Agile Software Development

23

23

Agile Principle 2

Welcoming change really means:

- Nobody gets in “trouble” when there’s a change.
- We’re all in this together. Everyone on the team, including the customers you’re collaborating with, owns the requirements and the changes to those requirements.

Agile Software Development

24

24

Agile Principle 2

Welcoming change really means:

- We don't sit on change until it's too late. we simply do our best to fix it as early as possible. So the damage is limited.
- We stop thinking of changes as mistakes.
- We learn from the changes.

Agile Software Development

25

25

Agile Principle 3:

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.



Agile Software Development

26

26

Agile Principle 3

- Traditional software teams are not very uncomfortable with the idea of welcoming changes.
- Simply accepting changes and welcoming them seems like a guaranteed way to introduce chaos to the project.
- How do agile teams manage to keep up with all of those changes while still dealing with the normal day-to-day issues ?

Agile Software Development

27

27

Agile Principle 3

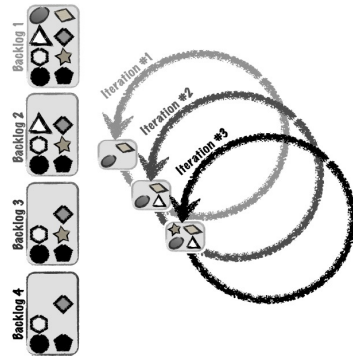
- Key to welcoming changes without introducing chaos lies in delivering working software frequently.
- Uses iteration to break the project into regular deadlines.
- During iterations, team delivers working software.
- At the end, review and retrospective is held.

Agile Software Development

28

28

Agile Principle 3



Agile Software Development

29

29

Agile Principle 3

- Then planning is done for the deliverables in the next iteration.
- Predictable schedule and the constant checkpoints help the team catch those changes early.
- Also gives everyone a blame-free environment where they can discuss each change and come up with a strategy to incorporate it into the project.

Agile Software Development

30

30

Agile Principle 3

- Time-boxed iterations give the project manager the control over deadline.
- This solves one of the project manager's biggest challenges: dealing with changes that come in very late in the project.
- Daily reviews and iteration retrospectives let the project manager enlist the whole team as eyes and ears to spot those changes much earlier.

Agile Software Development

31

31

Agile Principle 3

- Agile PM is working with the team to make sure that each person is constantly looking at the big picture and working toward the same goals.
- It's easier to do that when the team is working on a short iteration that will deliver working software.

Agile Software Development

32

32

Communicating and Working Together

- Software teams have been struggling with the “**how much documentation?**” question for as long as they’ve been developing software.
- Traditional thinking about software documentation is based on the idea of **traceability**.



Agile Software Development

33

33

Communicating and Working Together

- Comprehensive documentation actually leads to unnecessary changes and wasted effort.
- Agile practitioners should recognize that the traditional approach to documentation shares their goal.
- Agile has a different (and often much more efficient) approach to managing changes to your project.

Agile Software Development

34

34

Agile Principle 4:

The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

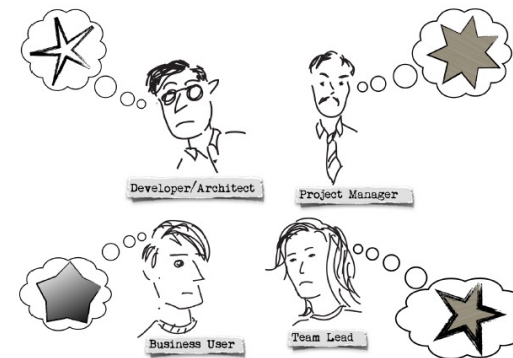


Agile Software Development

35

35

Agile Principle 4



Agile Software Development

36

36

Agile Principle 4

- Documentation is a good tool to accomplish the goals of communication.
- Goal is to make sure that the ideas in one's head match the ideas in another's head as closely as possible.
- Documentation is a good tool to accomplish that goal.
- But it's not the only communication tool available.

Agile Software Development

37

37

Agile Principle 4

- **Face-to-face communication** is almost always a better tool for sharing ideas within a software team than documentation.
- Talking through a problem in person is the most effective way to understand a new idea.
- It is much more likely to retain ideas that were expressed in conversation than by reading them on a document.

Agile Software Development

38

38

Agile Principle 4

- Agile communication practices focus most on individual people communicating with each other, and reserve documentation for those cases where complex information needs to be recalled in detail later.
- It's not hard for teams that are used to comprehensive documentation to move to more effective face-to-face communication.

Agile Software Development

39

39

Agile Principle 4

- Most efficient way for the team to communicate the important ideas needed for the project is to get everyone thinking the same way.
- When a group of people look at the world the same way, and communicate about both positive and negative ideas openly, they end up with a **shared perspective**.

Agile Software Development

40

40

Agile Principle 4

- Ultimate goal for team communication is to create a **sense of community** so that there's a lot of implied knowledge.
- Closer the team gets to having a sense of community and a shared perspective, the more often each person will independently come to a similar answer when posed the same question.

Agile Software Development

41

41

Agile Principle 5:

Business people and developers must work together daily throughout the project.



Agile Software Development

42

42

Agile Principle 5

- To build software well, teams need to have a lot of face-to-face discussion with business people.
- This is because they need to tap into their knowledge of the company's problems that are going to be solved with software.
- Business people got that knowledge by solving those same problems without software.

Agile Software Development

43

43

Agile Principle 5

- For most business people who work with software teams, the software project is typically a small part of their job.
- They don't want to spend all of their time with the software team.
- Therefore, software team would have very little contact with them.
- But team want as much contact with the business people as possible.

Agile Software Development

44

44

Agile Principle 5

- Agile teams are with an understanding that the software team is delivering value to the company.
- If that value is more than the cost to build the software, it's worth it for the company to invest in its development.
- Therefore business people can see that it's worth the effort they'll need to put in over the course of the project.

Agile Software Development

45

45

Agile Principle 5

- When business people and developers build software as a team, it's most efficient in the long run to work together daily throughout the project.
- Early involvement of business people means early feedback.
- It's much more expensive to make changes at the end of a project.

Agile Software Development

46

46

Agile Principle 5

- Therefore, the teams need to prioritize the most valuable features first.
- So that the business people can get that value delivered early.
- An agile team collaborates with the customer as someone with an equal say in the way the project is run.

Agile Software Development

47

47

Agile Principle 5

- A good product owner can help reduce the amount of time the business people spend with the team.
- Product owner can concentrate on understanding the value of the software and the business problem that needs to be solved.
- Team can use their face-to-face time with the business people to validate information learned from the product owner.

Agile Software Development

48

48

Agile Principle 6:

Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.



Agile Software Development

49

49

Agile Principle 6

- Projects run best when everyone in the company recognizes that the team is building valuable software, and when everyone on the team understands what makes the software valuable to the company.
- On the other hand, projects can break down in an environment where people don't see the value in the software, or where they aren't rewarded for building the software well.

Agile Software Development

50

50

Agile Principle 6

- Some companies set up performance reviews and compensation systems that discourage people from building software in an effective way.
- This can work against the project.
- Ultimately, everyone's performance should be based on what the team delivers, rather than the specific role that they played in it.

Agile Software Development

51

51

Agile Principle 6

- An good environment gives everyone on the team the support they need, and makes the project more successful.
- It will reward a
 - programmer who recognizes a part of the business problem that isn't being addressed and fixes it.
 - tester who sees a problem with the code or architecture and raises that with the team.

Agile Software Development

52

52

Agile Principle 6

- Comprehensive documentation and traceability matrices are a particularly insidious source of problems with a team's environment and support.
- This leads to a CYA attitude with in the team.
- Software team members need to CYA most in an environment where change is seen as a bad thing.

Agile Software Development

53

53

Agile Principle 6

- Opposite of CYA is trust.
- A company that develops only the minimal documentation needed for the project has an environment where the team is trusted to do the right thing when a change happens.
- Agile team with a "we're all in this together" attitude, where if the project fails everyone shares the blame, doesn't need to CYA.

Agile Software Development

54

54

Agile Principle 6

- It's easier for them to handle changes, because they don't need to maintain all of that unnecessary documentation.
- Instead, they can tackle the actual problem with face-to-face communication, and only write down what's necessary.
- They can do this knowing that the company trusts that they're doing the right thing even if it causes the project to take longer.

Agile Software Development

55

55

Project Execution

- Better communication and trust between everyone on the team is a great start.
- Once everyone gets along and knows how they fit into the project, they can start thinking about the biggest problem: actually doing the work every day.
- How does an agile team keep the project moving?

Agile Software Development

56

56

Agile Principle 7:

Working software is the primary measure of progress.



Agile Software Development

57

57

Agile Principle 7

- A good team works to make sure that everyone really understands where the project is at any time.
- But how do you actually communicate the status of a project?



Agile Software Development

58

58

Agile Principle 7

- A typical PM tries to keep the project on course and keeps everyone up to date with extensive schedules and status reports.
- But it's hard to capture the real essence of a project in a status report.
- So how do you report progress, if status reports aren't good enough?



Agile Software Development

59

59

Agile Principle 7

- Answer lies in working software.
- By delivering working software at the end of each iteration, and by doing a real product demonstration that shows everyone exactly what the team did.
- Thus they keep everyone up to date on the progress of the software in a way that is almost impossible to misread.

Agile Software Development

60

60

Agile Principle 8:

Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.



Agile Software Development

61

61

Agile Principle 8

- Hard-and-fast deadline is the primary tool in the command-and-control project manager's toolbox.
- Whenever a deadline is approaching, everyone turns to nights and weekends as the first resort.
- This can be due to an unrealistic deadline.
- In the long run, this approach doesn't work.

Agile Software Development

62

62

Agile Principle 8

- An agile teams believe in maintaining a sustainable pace.
- They plan on delivering work that can actually be done in the time they reserve for it.
- Iterative development makes it more realistic, as it is easier to estimate for a shorter period.
- By promising to deliver only what they can build, team can deliver within working hours.

Agile Software Development

63

63

Agile Principle 9:

Continuous attention to technical excellence and good design enhances agility.



Agile Software Development

64

64

Agile Principle 9

- Poor technical design and coding can be a reason for working extra hours.
- Tracking down bugs and patching up code takes a considerable amount of time.
- It's a lot faster in the long run to avoid a bug now than it is to fix it later.
- It's a lot easier to maintain code that's well designed.

Agile Software Development

65

65

Agile Principle 9

- Novel approaches have given developers patterns and tools to bring technical excellence to every project.
 - Object oriented design and analysis
 - Design patterns
 - Decoupled and Service-oriented architecture
- But agile teams do not spend a lot of time creating large-scale designs at the start of every software project.

Agile Software Development

66

66

Agile Principle 9

- Instead, Agile developers build up great coding habits that help them create well-designed code.
- They're constantly on the lookout for design and code problems, and take the time to fix those problems as soon as they're discovered.
- This creates a codebase that's easy to maintain.

Agile Software Development

67

67

Constantly Improving Project & Team

- One of the most basic design principles is "Keep it simple, Stupid" (KISS).
- An agile team lives by this principle in the way they plan the project, build the software, and run the team.



Agile Software Development

68

68

Agile Principle 10:

Simplicity—the art of maximizing the amount of work not done—is essential.



Agile Software Development

69

69

Agile Principle 10

- Adding code to an existing project often makes it more complex.
- Dependencies between systems, objects, services, etc., make code more complex and difficult to change.
- Using iteration and building the minimal documentation at the start of the project helps software team avoid delivering unnecessary software.

Agile Software Development

70

70

Agile Principle 10

- But developers might feel like it's too early to start the code for a project until they've made a lot of the design and architecture decisions and written them down.
- Otherwise the team will write code today that they'll need remove tomorrow when the design changes.

Agile Software Development

71

71

Agile Principle 10



Agile Software Development

72

72

Agile Principle 10

- Deleting code is not particularly destructive, because it can be recovered from the version control system.
- Most destructive thing is to build new code, and adding dependencies.
- This may eventually results with an unmaintainable mess of spaghetti code.

Agile Software Development

73

73

Agile Principle 10

- Maximizing the amount of work not done means avoiding this mess.
- This is by building systems that don't have a lot of dependencies and unnecessary code.
- If a feature is not valuable, it's often cheaper for the company in the long run to not build it at all.

Agile Software Development

74

74

Agile Principle 11:

Best architectures, requirements, and designs emerge from self-organizing teams.



Agile Software Development

75

75

Agile Principle 11

- Overly complex designs are common in teams that do too much up-front planning.
- In waterfall process, there's an entire phase dedicated to requirements, and another phase dedicated to design and architecture.
- So a traditional team try to come up with them most awesome architecture at the end of the phase.

Agile Software Development

76

76

Agile Principle 11

- A self-organizing team, does not have an explicit requirements or design phase.
- When a team is self-organizing, it means that they work together to plan the project.
- They continually come together as a team to revise that plan.

Agile Software Development

77

77

Agile Principle 11

- They break the project down into user stories or other small chunks.
- Then start working on the ones that deliver the most value to the company first.
- Finally do they start thinking about detailed requirements, design, and architecture.
- This makes the traditional software architect's job more difficult, but also more satisfying.

Agile Software Development

78

78

Agile Principle 11

- On an agile team, every member shares responsibility for the architecture.
- A senior software architect or designer still has an important role to play.
- In fact, when a team is building the software piece by piece, starting with the most valuable chunks, the architect's job becomes more challenging.

Agile Software Development

79

79

Agile Principle 11

- Instead of creating one big design at the beginning of the project that covers all of the requirements, agile architects use **incremental design**.
- This involves techniques that allow them to design a system that is not just complete, but also easy for the team to modify as the project changes.

Agile Software Development

80

80

Agile Principle 12:

At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.



Agile Software Development

81

81

Agile Principle 12

- Agile teams constantly inspect and adapt.
- That is they look at how their projects have run and they use that knowledge to improve in the future.
- They do it every day from the beginning of the project.
- Team members need be honest with themselves and their teammates about what's working and what isn't.

Agile Software Development

82

82

Agile Principle 12

- But looking back and taking stock of what went right and wrong is one of those things that many teams genuinely intend to do, but rarely actually get around to doing.
- Over time, people on the teams get more and more comfortable talking about these things.
- Eventually, everyone starts to see this as more constructive than critical.

Agile Software Development

83

83

Agile Principle 12

- Another reason that teams don't do this reflection is that they never set aside the time to do it.
- Teams that start each project by reserving time both at the end of each iteration and at the end of the project to meet, review, assess, and come up with a plan for improvement are more likely to actually meet and talk about how they did.

Agile Software Development

84

84

Bringing All the Principles Together

- Teams that use agile more effectively recognize that the practices aren't just a "menu" to choose from.
- Key to using these practices together is the mindset that the team brings to the project.
- That mindset is driven by the agile values and principles.

Agile Software Development

85

85

Bringing All the Principles Together

- Agile is different because it starts with values and principles.
- A team going agile has to look honestly not just at the way they build software, but also at the way they interact with each other and the rest of their company.



Agile Software Development

86

86

Bringing All the Principles Together

- By first understanding the principles, and only then applying a methodology an agile team can be realistic about finding better ways to run their projects.
- This gives them a real path to increased agility, and lets them build and deliver better software.

Agile Software Development

87

87